Standalone Static Data Extraction Program

Revised Development & Architecture Plan — Aligned with ASignX v2.0 Rebuilt System

Project: ASignX Machine Learning (v2.0) | Core Engine: `ml/extract_dataset.py` (Implemented) | Target: `ml/data/landmarks.csv` & `SignResMLP`

1. Executive Summary & Alignment with the Current Rebuilt System

This revised development plan documents the architecture, mathematical filtering pipeline, and portable deployment roadmap of the **ASignX Static Hand-Sign Data Extraction Program**, updated to reflect the **rebuilt ASignX v2.0 Machine Learning system**.

In the current build, the core extraction, canonicalization, 4-pass outlier/duplicate cleaning, session synthesis, and non-destructive CSV append engine has been fully implemented in `ml/extract_dataset.py` and verified by automated unit and integration tests in `tests/test_upgraded_architecture.py`. It connects directly to the rebuilt **91D Kinematic** `SignResMLP` classifier (`ml/model.py` , `ml/train.py`), the `CosinePrototypeHead` for zero-retrain sign addition, and the **63D** `class_centroids` exporter used by **ASignLearn**.

> **What Is Built Now vs. Portable Expansion**
>
> - **Implemented Core Module (`ml/extract_dataset.py`):** Accepts any folder hierarchy or `.zip` dataset upload, normalizes folder labels case-insensitively, optionally consolidates accepted images into a staging folder (`--staging-dir`), runs a 4-pass duplicate/outlier cleaner, synthesizes ≥ 5 independent sessions per sign, and atomically appends to `landmarks.csv` .
> - **Seamless Compatibility with 91D Kinematic `SignResMLP` :** Writes standard 63D wrist-normalized coordinates (`PREPROCESSING_VERSION = "wrist_max_radius_v1"`) to `landmarks.csv` while `TorchKinematicEnricher` inside `SignResMLP` automatically expands each 63D row into 91 biomechanical features (joint flexion cosines, fingertip distances, and 3D palm normal) on the fly.
> - **Next-Phase Portable UI Wrapper:** Wraps `extract_and_clean_dataset()` into a standalone Drag-and-Drop Web/Desktop uploader and single-folder executable for non-technical users on any device.

2. How the Extractor Solves the 3-Way Split Requirement in `ml/dataset.py`

In `ml/dataset.py` , `create_splits()` partitions static data into **Train (70%)**, **Validation (15%)**, and **Test (15%)** by grouping rows according to their `session_id` , and `validate_splits()` enforces two strict invariants:

1. Every sign label must have at least **3 independent `session_id` values** (otherwise raising `ValueError: <Label>: need at least 3 independent sessions`).
2. No two splits may share an identical 63D feature vector `sample.features.tobytes()` (otherwise raising `ValueError: Identical features across splits; remove copied/duplicate recordings`).

`ml/extract_dataset.py` solves both constraints automatically via `assign_balanced_sessions()` and Pass 1/Pass 4 deduplication:

- **Automatic Balanced Session Synthesis (`assign_balanced_sessions`):** For every canonical label, cleaned candidates are partitioned into `num_sessions = max(min_sessions, ceil(N / target_chunk_size))` independent

sessions (with `min_sessions = 5` by default, always enforced ≥ 3, and `target_chunk_size = 40`). Each session receives a deterministic tag: `ext_<source_tag>_<LABEL>_s0000`, `ext_<source_tag>_<LABEL>_s0001`, etc.

- **Cross-Split Duplicate Prevention:** By deduplicating both exact file hashes (Pass 1) and near-identical feature vectors within an ε-ball against existing CSV rows and within the new batch (Pass 4), `validate_splits()` is guaranteed never to fail on duplicate features across splits.

3. Flexible Dataset Discovery & Staging Consolidation (`discover_static_dataset`)

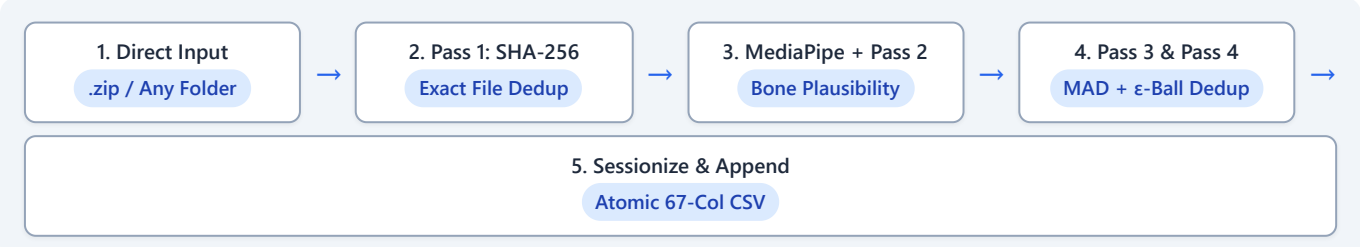
Your proposed idea of extracting images from folders like `'A'` or `'a'` and placing them into a clean, canonical accepted folder per sign is supported alongside a fast zero-copy virtual crawler inside `discover_static_dataset(root_dir, staging_dir=None)`:

Ingestion Mode	How It Works in `ml/extract_dataset.py`	When to Use
Default Mode: Zero-Copy Virtual Crawler (`--staging-dir` omitted)	Recursively scans arbitrary directories or extracts `.zip` archives into a temporary workspace, infers canonical labels via `infer_label_from_path()`, deduplicates via SHA-256, and streams directly into MediaPipe.	Fastest extraction on laptops, workstations, or Colab with zero permanent image duplication.
Staging Consolidation Mode (`--staging-dir <path>` provided)	Automatically copies every unique, accepted source image into a clean, canonical folder structure: `<staging_dir>/<LABEL>/<sha256_12>.jpg` before running MediaPipe landmark extraction.	When you want to merge multiple messy datasets into one clean, human-inspectable image folder per sign.

3.1 Implemented Label Canonicalization & Wrapper-Folder Filtering

- **Wrapper Directory Filtering (`IGNORED_DIR_NAMES`):** Automatically ignores structural folders including `train`, `training`, `val`, `valid`, `validation`, `test`, `testing`, `images`, `dataset`, `data`, `raw`, `processed`, `static`, `hands`, `asl`, `fsl`, `asl_alphabet_train`, and `asl_alphabet_test`.
- **Prefix & Case Normalization (`canonicalize_label`):** Strips prefixes matching `^(?:sign|class|label|letter|char|folder)[\s_\-]+` case-insensitively and normalizes spaces/hyphens to uppercase (e.g., `"a"` → `"A"`, `"sign_b"` → `"B"`, `"Class-Hello"` → `"HELLO"`).
- **Filename Stem Fallback:** If images are placed directly in a root folder without class subfolders (e.g., `A001.jpg`, `B002.png`), `infer_label_from_path()` automatically extracts the leading alphabetic stem.

4. Implemented 4-Pass Extraction & Outlier/Duplicate Cleaning Pipeline



Implemented Pass	Function in `ml/extract_dataset.py`	Mathematical / Geometric Criterion	Report Counter Updated
Pass 1: Exact File Hash Deduplication	`discover_static_dataset()`	Computes `hashlib.sha256(img_bytes)` during folder discovery; skips byte-identical files before invoking OpenCV or MediaPipe.	`exact_duplicates_skipped`
Pass 2: Anatomical Bone-Segment Plausibility	`check_bone_plausibility()`	Validates `wrist_max_radius_v1` invariants via `validate_features()` and checks all 20 bone segments (`BONE_SEGMENTS`) satisfy $0.02 \le \|\|\mathbf{p}_{end} - \mathbf{p}_{start}\|\|_2 \le 0.85$.	`anatomical_outliers_rejected`
Pass 3: Intra-Class Robust MAD Outlier Rejection	`filter_statistical_outliers_mad()`	Computes class median pose $\boldsymbol{\mu}_c$ = median($\mathbf{F}_c$), Euclidean distances $d_i = \|\|\mathbf{f}_i - \boldsymbol{\mu}_c\|\|_2$, and robust scale $\sigma_{MAD} = 1.4826 \cdot \text{median}(\|d_i - \text{median}(d)\|)$. Rejects samples with $d_i > \text{median}(d) + 3.5\ \sigma_{MAD}$.	`statistical_outliers_rejected`
Pass 4: Cross-CSV & Intra-Batch ε-Ball Pruning	`prune_near_duplicates()`	Compares each candidate $\mathbf{f}$ against existing vectors for that label in `landmarks.csv` and kept batch vectors; prunes if min $\|\|\mathbf{f} - \mathbf{g}\|\|_2 < \varepsilon$ (default $\varepsilon = 0.015$).	`already_in_csv_skipped` & `near_duplicates_pruned`

5. Non-Destructive Atomic CSV Append & Downstream Model Synergy

5.1 Atomic 67-Column Schema Compliance (`append_to_landmarks_csv`)

To guarantee that existing training data inside `ml/data/landmarks.csv` is never overwritten or corrupted, `append_to_landmarks_csv()` writes to a temporary file (`tempfile.mkstemp`) in the same directory:

1. Writes the exact 67-column header required by `ml/data_io.py` : `CSV_COLUMNS = ["label", "session_id", "handedness", "preprocessing", "x0", "y0", "z0", ..., "x20", "y20", "z20"]` .
2. Streams all existing rows from `ml/data/landmarks.csv` verbatim into the temporary file.
3. Appends the newly cleaned and sessionized rows with `PREPROCESSING_VERSION = "wrist_max_radius_v1"` and 8-decimal float precision (`f"{val:.8f}"`).
4. Executes an atomic OS-level file replacement (`os.replace(temp_path, csv_path)`) and automatically deletes stale `ml/models/*.splits.json` manifests so the next `python -m ml.train` run cleanly rebuilds train/validation/test splits.

5.2 How Extracted Data Feeds the Rebuilt v2.0 AI Architecture

- **Path A — Full Retraining with 91D Kinematic `SignResMLP` (`python -m ml.train`):** Reads `landmarks.csv`, applies 3D geometric augmentation (`augment_landmarks_3d`) on the training split, enriches each 63D vector to 91D inside `TorchKinematicEnricher`, trains with `AdamW` + `CosineAnnealingLR` + label smoothing (`0.05`), and exports 63D `class_centroids` for **ASignLearn** per-finger coaching.
- **Path B — Instant Zero-Retrain Sign Registration (`SignPredictor.register_new_sign`):** Because `SignResMLP` uses a metric-learning `CosinePrototypeHead`, newly extracted samples for a brand-new sign can also be registered directly into an existing `sign_classifier.pth` checkpoint in milliseconds without retraining existing classes.

6. Current Codebase Structure & Command-Line Usage

```
ASignX-Machine-Learning/
 ├── asignx/
 │   ├── detector.py                # MediaPipe Hands wrapper (static_image_mode=True for extraction)
 │   └── visualizer.py              # Real-time HUD overlay with live Sign Recognition Card
 ├── ml/
 │   ├── extract_dataset.py         # [IMPLEMENTED] Standalone flexible dataset extractor & 4-pass cleaner
 │   ├── data_io.py                 # 67-column CSV schema (CSV_COLUMNS) & reader/writer
 │   ├── preprocessing.py           # wrist_max_radius_v1, 91D kinematic enricher & 3D augmentation
 │   ├── model.py                   # SignResMLP + TorchKinematicEnricher + CosinePrototypeHead
 │   ├── train.py                   # Upgraded AdamW + CosineAnnealingLR + class_centroids trainer
 │   └── export_onnx.py             # Web deployment exporter (.onnx, .meta.json, .weights.json)
 ├── web_client/
 │   └── asignx_client_sdk.ts       # Browser SDK for ASignLearn coaching & ASignLive recognition
 └── tests/
     └── test_upgraded_architecture.py # Automated test suite verifying ml/extract_dataset.py
```

CLI Usage in the Current Build

```
# 1. Extract from any folder structure or .zip archive and safely append to landmarks.csv
python -m ml.extract_dataset ^
    --input "D:/Datasets/ASL_Upload.zip" ^
    --output "ml/data/landmarks.csv" ^
    --mad-threshold 3.5 ^
    --dedup-epsilon 0.015 ^
    --min-sessions 5

# 2. Optional: Consolidate accepted images into a clean staging folder per sign (<staging>/<LABEL>/)
python -m ml.extract_dataset ^
    --input "D:/Datasets/Messy_Folders" ^
    --output "ml/data/landmarks.csv" ^
    --staging-dir "ml/data/staged_images"

# 3. Train the rebuilt 91D Kinematic SignResMLP on the updated landmarks.csv
python -m ml.train --data ml/data/landmarks.csv --output ml/models/sign_classifier.pth --epochs 50 --batch-size 64
```

7. Implementation Status & Next-Phase Portable Packaging Roadmap

Phase	Component / File	Status	Description
Phase 1: Structure-Agnostic Discovery & Zip Ingestion	`discover_static_dataset()` `infer_label_from_path()`	**COMPLETED**	Recursive scanner supporting flat, nested (`train/val/test`), and `.zip` archives, case-insensitive label canonicalization, SHA-256 exact deduplication, and optional `--staging-dir` consolidation.
Phase 2: MediaPipe Static Extraction & Bone Filter	`extract_and_clean_dataset()` `check_bone_plausibility()`	**COMPLETED**	Extracts 21 3D hand landmarks, applies `normalize_landmarks()` , and filters collapsed or implausible skeletons across all 20 finger bone segments.
Phase 3: Statistical MAD & ε-Ball Deduplication	`filter_statistical_outliers_mad()` `prune_near_duplicates()`	**COMPLETED**	Per-class Median Absolute Deviation (MAD) outlier rejection and greedy ε-ball near-duplicate pruning against both existing CSV rows and new batch candidates.
Phase 4: Auto-Sessionizer & Atomic CSV Merger	`assign_balanced_sessions()` `append_to_landmarks_csv()`	**COMPLETED**	Synthesizes ≥ 5 independent sessions per sign, performs atomic temp-file append to `landmarks.csv` , and invalidates stale `*.splits.json` manifests.
Phase 5: Portable GUI Uploader & Standalone Executable	`extractor_ui.py` / PyInstaller Bundle	**NEXT OPTIONAL EXPANSION**	Lightweight drag-and-drop desktop/browser UI wrapping `extract_and_clean_dataset()` and displaying live `ExtractionReport` counters on any standalone device.